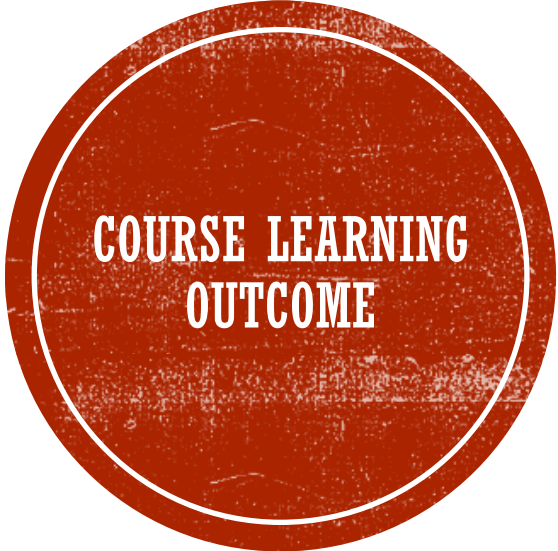


CHAPTER 1 (PART 1) : BASIC OF PYTHON PROGRAMMING

DFK50083 PYTHON PROGRAMMING
SESSION I : 2026/2027



COURSE LEARNING OUTCOME

Display general description about Python

Display the reason of using Python

Display Basic Principles of Python

Display the features of Python

Installing and Python
Environment

- Show the requirements for installing Python
- Perform Installation of Python
- Verify the Python installation

- “It is general purpose programming language. It is similar to other programming languages that you might have heard of such as C++, JavaScript or Microsoft’s C# and Oracle’s Java”



PYTHON PROGRAMMING

- Python is an **interpreted, object-oriented, high-level** programming language with **dynamic semantics**.
 - Its **high-level built in data structures**, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together.
 - Python's **simple, easy to learn syntax** emphasizes **readability** and therefore reduces the cost of program maintenance.
 - Python supports modules and packages, which encourages program modularity and code reuse.
-
- Source: <https://www.python.org/doc/essays/blurb/> (2021)



WHY PYTHON PROGRAMMING ?

1. Its flexibility and simplicity which makes it easy to learn.
2. Its use by the Data Science community where it provides a more standard programming language than some rivals such as R.
3. Its suitability as a scripting language for those working in the DevOps field where it provides a higher level of abstraction than alternative languages traditionally used.
4. Its ability to run on (almost) any operating system, but particularly the big three operating systems Windows, MacOS and Linux.
5. The availability of a wide range of libraries (modules) that can be used to extend the basic features of the language.
6. It is Free!



VERSION OF PYTHON

- Currently there are two main versions of Python called Python 2 and Python 3.
- Python 2 was launched in October 2000 and has been, and still is, very widely used.
- Python 3 was launched in December 2008 and is a major revision to the language that is not backward compatible.
- The issue between the two versions can be highlighted by the simple print facility:
 - In Python 2 this is written as `print 'Hello World'`
 - **In Python 3 this is written as `print ('Hello World')`**
- Python is an example of a hybrid programming language as it allows you to write very procedural code, to use objects in an object oriented manner and to write functional programs



BASIC PRINCIPLE OF PYTHON(1)

1. Hybrid programming language

- The code can be written in a procedural and object-oriented manner.

2. Interpreted language

- The source code of a Python is converted into bytecode that is then executed by the Python virtual machine. There is no explicit compilation step in Python.

3. Dynamically Typed Language

- The data type checking is performed at runtime. It doesn't know about the type of the variable until the code is run.
- In Python, the variable is declared without the need to specify the data type.



BASIC PRINCIPLE OF PYTHON(2)

4. Automatic memory management and garbage collection

- The garbage collector automatically frees up the memory which was allocated by the program but is no longer referenced.

5. Can be used for many types of applications:

- Web Development
- Python for Scientific development
- Data Science and Analysis

6. Can be run using several ways:

- Interactively using the Python interpreter
- Stored in a file and run using the Python command
- From within a Python IDE (Integrated Development Environment) such as PyCharm.



FEATURES OF PYTHON

1. Easy To Learn and Use
2. Expressive Language
3. Interpreted Language
4. Cross Platform Language
5. Free and Open Sources
6. Object oriented Language
7. Extensible
8. Large standard library
9. GUI Programming Support
10. Integrated
11. Embeddable

<https://www.javatpoint.com/python-features>



HOW TO RUN PYTHON PROGRAM ?

There are several ways in which you can run a Python program, including

- Interactively using the Python interpreter
- Stored in a file and run using the Python command
- Run as a script file specifying the Python interpreter to use within the script file
- **From within a Python IDE (Integrated Development Environment) such as PyCharm.**



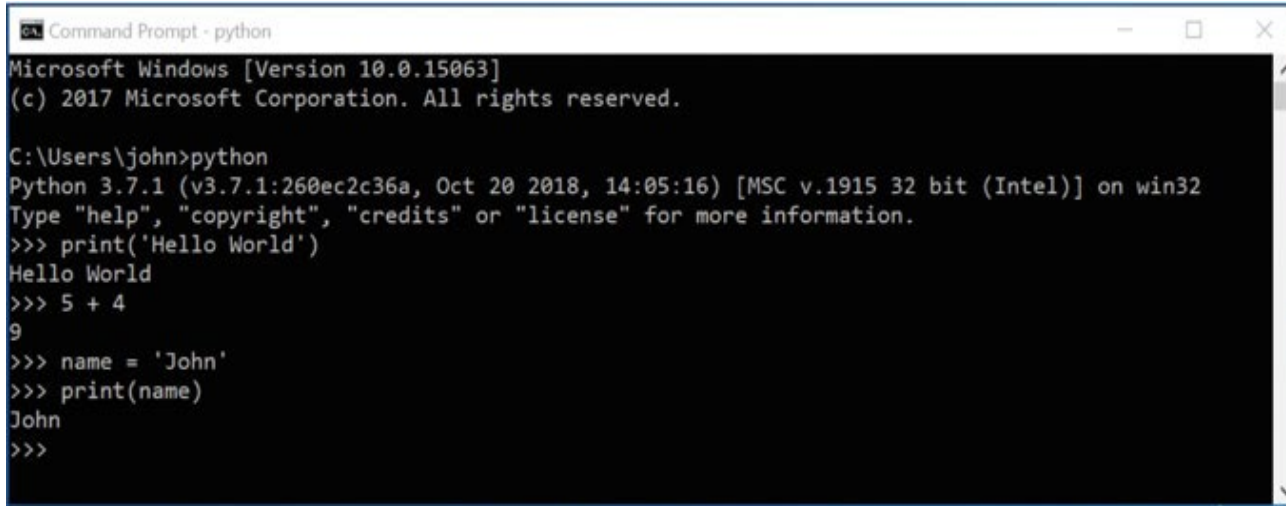
INTERACTIVELY USING THE PYTHON INTERPRETER

- This method uses the Python REPL (Read Evaluate Print Loop style of operation).
- Using the REPL, Python statements and expressions can be typed into the Python prompt and will then be executed directly.
- The values of variables will be remembered and may be used later in the session.
- To run the Python REPL, Python must have been installed onto the computer system you are using.
- Once installed you can open a **Command Prompt window (Windows)** or a Terminal window (Mac) and type python into the prompt.



INTERACTIVELY USING THE PYTHON INTERPRETER

- This is shown for a Windows machine below:



```
Command Prompt - python
Microsoft Windows [Version 10.0.15063]
(c) 2017 Microsoft Corporation. All rights reserved.

C:\Users\john>python
Python 3.7.1 (v3.7.1:260ec2c36a, Oct 20 2018, 14:05:16) [MSC v.1915 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print('Hello World')
Hello World
>>> 5 + 4
9
>>> name = 'John'
>>> print(name)
John
>>>
```

In this case we

Printed out the string 'Hello World'.

Added 5 and 4 together and got the result 9.

Stored the string 'John' in a variable called name.

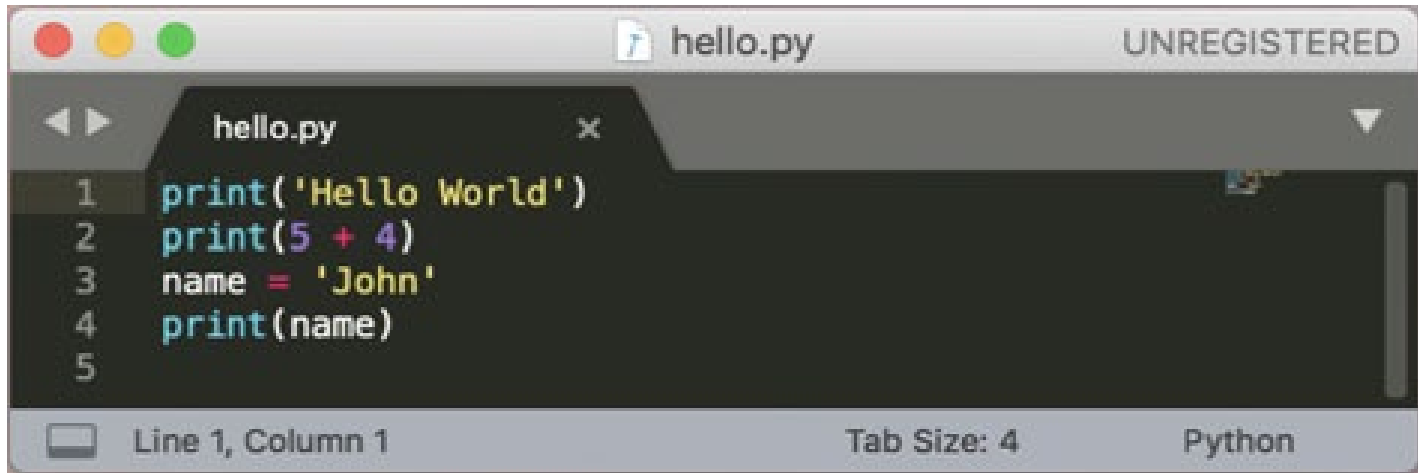
Printed out the contents of the variable name.

In the above example, we interactively typed in several Python commands and the Python interpreter 'Read' what we have typed in, 'Evaluated' it (worked out what it should do), 'Printed' the result and then 'Looped' back ready for further input.



RUNNING A PYTHON FILE

- We can of course store the Python commands into a file.
- This creates a program file that can then be run as an argument to the python command.
- For example, given a file containing the following file (called hello.py) with the 4 commands in it:



The screenshot shows a code editor window titled 'hello.py' with 'UNREGISTERED' in the top right corner. The editor has a dark background and shows the following code:

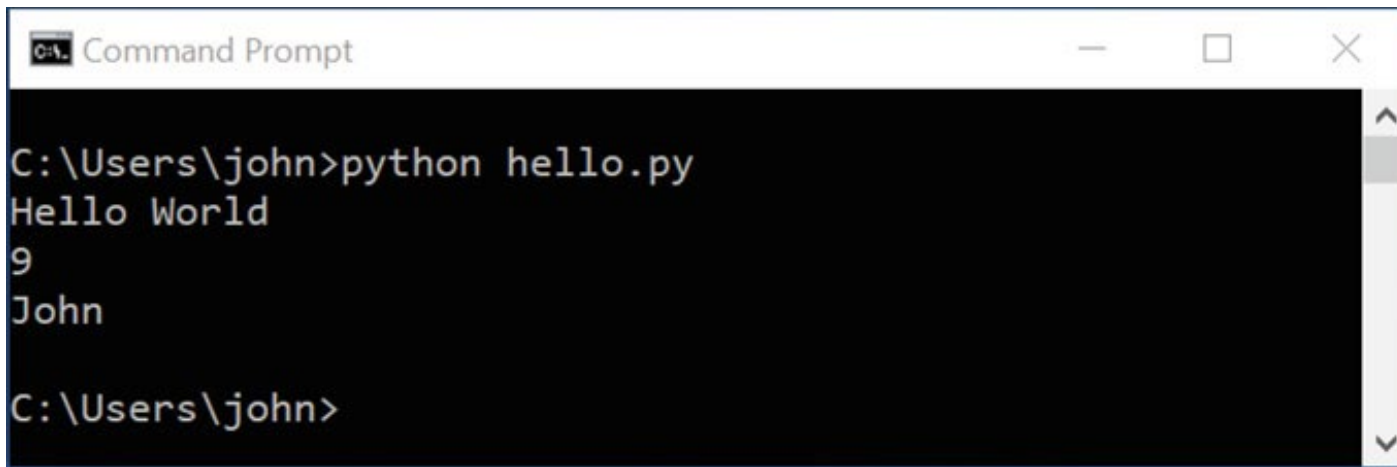
```
1 print('Hello World')
2 print(5 + 4)
3 name = 'John'
4 print(name)
5
```

The status bar at the bottom indicates 'Line 1, Column 1', 'Tab Size: 4', and 'Python'.



RUNNING A PYTHON FILE

- To run the hello.py program on a PC using Windows we can use the python command followed by the name of the file:



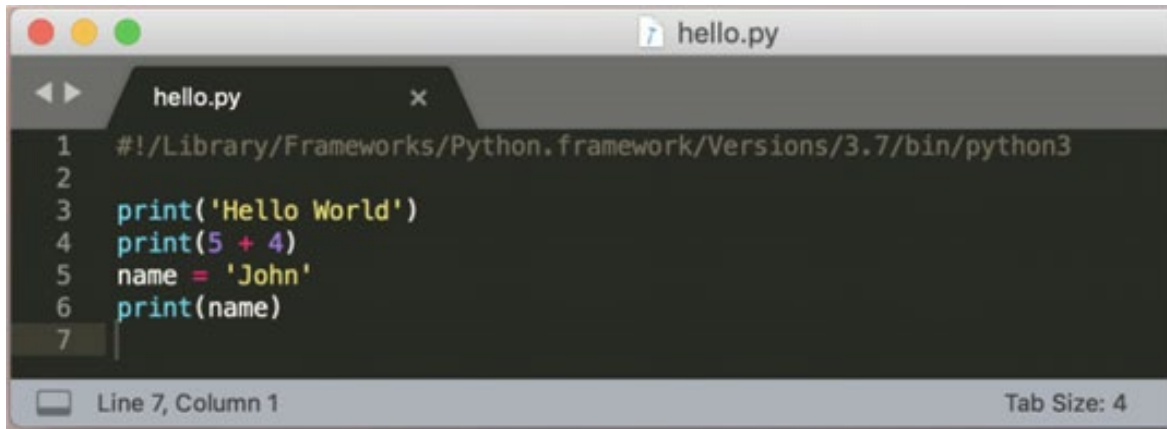
A screenshot of a Windows Command Prompt window. The title bar at the top reads "C:\ Command Prompt" and includes standard window controls (minimize, maximize, close). The command prompt shows the following sequence of text: the prompt "C:\Users\john>" followed by the command "python hello.py", the output "Hello World", a blank line, the output "9", another blank line, the output "John", and finally the prompt "C:\Users\john>" again. A vertical scrollbar is visible on the right side of the command prompt window.

```
C:\Users\john>python hello.py
Hello World
9
John
C:\Users\john>
```



EXECUTING A PYTHON SCRIPT

- A script is a stand-alone file that can be run directly without the need to (explicitly) use the python command.
- This is done by adding a special line to the start of the Python file that indicates the Python command (or interpreter) to use with the rest of the file.
- This line must start with `'#!'` and must come at the start of the file.



The screenshot shows a code editor window with a single tab titled 'hello.py'. The code inside the tab is as follows:

```
1 #!/Library/Frameworks/Python.framework/Versions/3.7/bin/python3
2
3 print('Hello World')
4 print(5 + 4)
5 name = 'John'
6 print(name)
7
```

The status bar at the bottom of the editor indicates 'Line 7, Column 1' and 'Tab Size: 4'.



EXECUTING A PYTHON SCRIPT

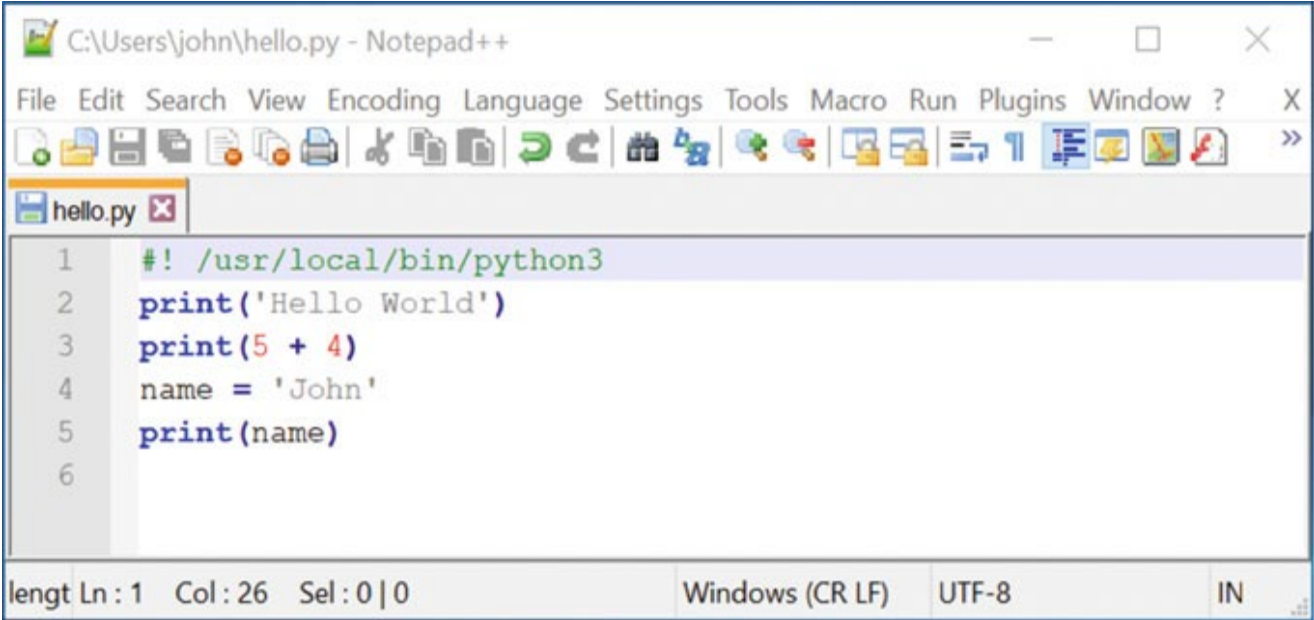
- Different systems will store Python in different locations and thus might need different first lines, for example on a Linux we might write:

```
#!/usr/local/bin/python3  
  
print('Hello, world')  
  
print(5 + 4)  
  
name = 'John'  
  
print(name)
```



EXECUTING A PYTHON SCRIPT

1. An example of the hello.py script for a Windows or Linux machine is given below using Notepad++ on a Windows box.



The image shows a screenshot of the Notepad++ text editor. The title bar at the top reads "C:\Users\john\hello.py - Notepad++". The menu bar includes File, Edit, Search, View, Encoding, Language, Settings, Tools, Macro, Run, Plugins, Window, and a help icon. Below the menu bar is a toolbar with various icons for file operations and editing. The editor window shows a single file named "hello.py" with the following Python code:

```
1  #!/usr/local/bin/python3
2  print('Hello World')
3  print(5 + 4)
4  name = 'John'
5  print(name)
6
```

The status bar at the bottom of the window displays "lengt Ln : 1 Col : 26 Sel : 0 | 0", "Windows (CR LF)", "UTF-8", and "IN".



USING PYTHON IN AN IDE

1. We can also use an IDE such as PyCharm to writing and execute our Python program. The same program is shown using PyCharm below:

